Implementing Dynamic Auto Scaling on AWS
Using Target Tracking Policy

Overview

This tutorial covers configuring a **Target Tracking Scaling Policy** on AWS, which automatically adjusts the number of EC2 instances based on CPU utilization. Instead of a fixed capacity that stays constant regardless of load, this setup reacts to real time demand, scaling out under load and back in once it subsides, within a defined capacity ceiling that keeps cost predictable.

Why Target Tracking Scaling?

A fixed capacity Auto Scaling Group keeps a set number of instances running at all times, useful for predictable, steady workloads, but wasteful during low traffic and risky during unexpected spikes, since it can't respond on its own.

Target Tracking Scaling solves this by using **CloudWatch metrics** (like average CPU utilization) to automatically adjust capacity toward a target value you define. It helps balance performance and cost, scaling out only when demand actually increases, and scaling back in once it drops, rather than running excess instances around the clock.

Technology Stack

Amazon EC2	Compute Hosting
Auto Scaling Group	Instance Management
Launch Template	Instance Configuration
Application Load Balancer	Traffic Distribution
Amazon CloudWatch	Monitoring
Amazon Linux	Operating System
stress	Load Testing

Objectives

- Configure a Target Tracking Scaling Policy
- Monitor Auto Scaling behavior using CloudWatch metrics
- Simulate application load using stress
- Verify automatic scale-out and scale-in
- Understand how Target Tracking helps reduce over-provisioning

Prerequisites

- AWS Account
- Basic knowledge of Launch Templates and Auto Scaling
- Familiarity with EC2 and Application Load Balancers

Note: *This tutorial recreates the infrastructure required to demonstrate Target Tracking Scaling. For the basic structure of Launch Templates, Auto Scaling Groups, and Load Balancers, refer to the Auto Scaling tutorial.*

Step 1: Create the Launch Template

A Launch Template defines how new EC2 instances get built automatically, so the Auto Scaling Group knows what to launch.

- Go to **Amazon EC2 → Launch Templates → Create Launch Template**
- Configure the template:
    - **Name:** ec2-launch-template
    - **AMI:** Amazon Linux
    - **Instance type:** t3.micro (or available alternative)
    - **Key pair:** Select existing or create a new one
    - **Security group:** allow HTTP (80), SSH (22)
    - **User data:**

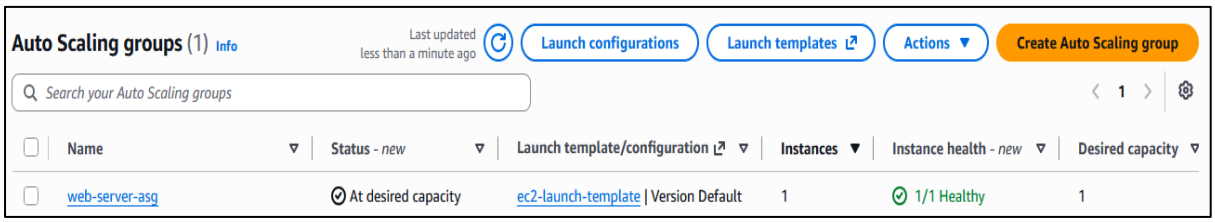
```
#!/bin/bash

sudo su

dnf install httpd -y

systemctl start httpd

echo "<h1>Target Tracking Auto Scaling Demo</h1> <p>This EC2 instance was launched automatically by an Auto Scaling Group.</p> <p>Scaling is managed using a Target Tracking Policy based on CPU Utilization.</p>" > /var/www/html/index.html
```

Step 2: Create the Auto Scaling Group

This is where the actual scaling behavior starts to take shape, the group manages how many instances run at any time.

- Go to **Auto Scaling Groups → Create Auto Scaling Group**

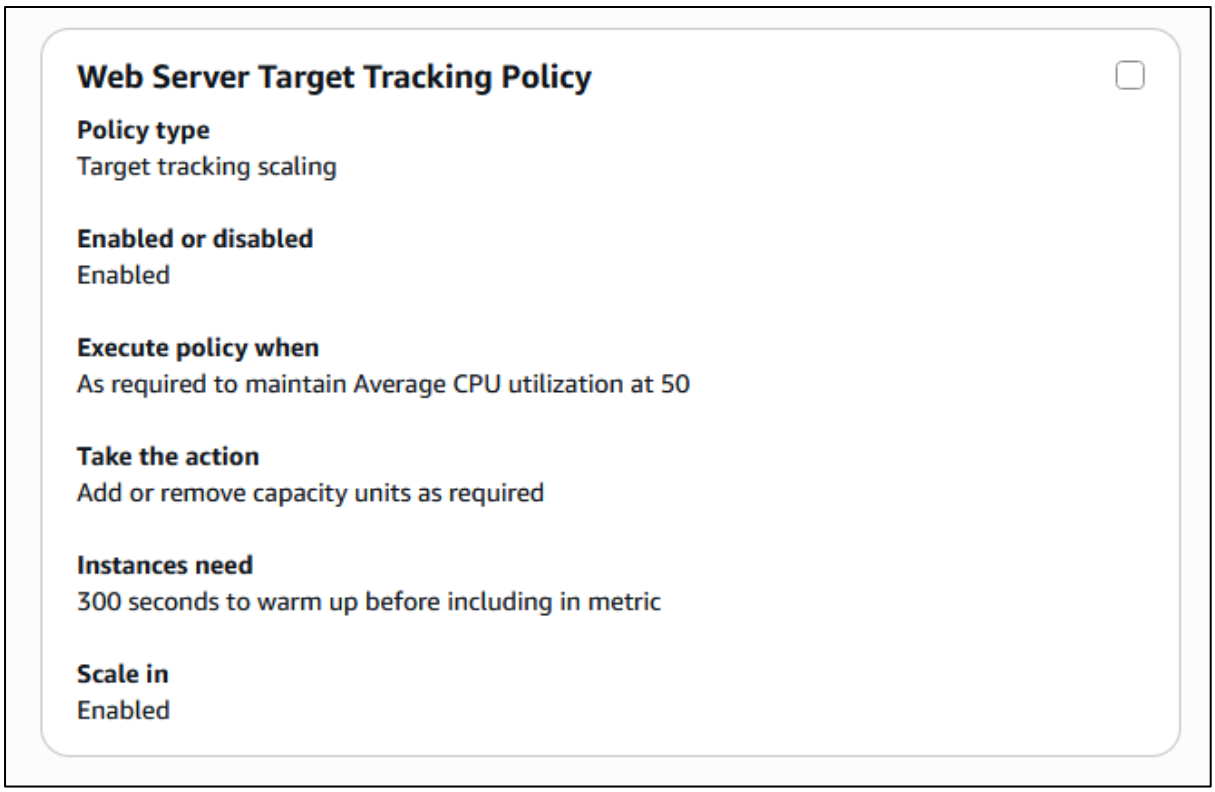
- Configure the following:
    - **Name:** `web-server-asg`
    - **Launch Template:** Select the launch template created in **Step 1**.
    - **VPC/Subnets:** Select your preferred VPC and at least two Availability Zones.
    - Under **Load Balancing**, choose **Attach to a new Application Load Balancer**.
    - **Scheme:** Select **Internet-facing**.
    - Create a new **Target Group** and use the default **HTTP (Port 80)** listener.
    - **Minimum Capacity:** `1`
    - **Desired Capacity:** `1`
    - **Maximum Capacity:** `2` (or `3`)



Step 3: Configure the Target Tracking Scaling Policy

The target tracking policy watches CPU usage and adjusts capacity to stay near the target you set.

- Open the Auto Scaling Group → **Automatic scaling tab**
- Click **Create dynamic scaling policy**
- Configure:
    - **Policy type:** Target Tracking
    - **Metric:** Average CPU Utilization
    - **Target value:** 50%
    - **Instance warm-up:** default
    - **Disable scale-in:** unchecked



Step 4: Deploy and Verify

Before testing scaling, confirm the base setup actually works.

- Navigate to **EC2 → Load Balancers**, copy the DNS name
- Open the DNS name in a browser to confirm the app loads

- Check the Target Group to confirm targets are healthy.

Step 5: Generate CPU Load

- Connect to a running instance
- Install the load testing tool

```
$ sudo yum install stress -y
```

```
Last metadata expiration check: 0:06:01 ago on Thu Jul  9 09:03:43 2026.
Dependencies resolved.
================================================================================
 Package          Architecture          Version                    Repository
================================================================================
Installing:
 stress           x86_64                1.0.7-2.amzn2023.0.1       amazonlinux

Transaction Summary
================================================================================
Install  1 Package

Total download size: 34 k
Installed size: 68 k
Downloading Packages:
stress-1.0.7-2.amzn2023.0.1.x86_64.rpm                      986 kB/s |
--------------------------------------------------------------------------------
Total                                                       457 kB/s |
Running transaction check
Transaction check succeeded.
```

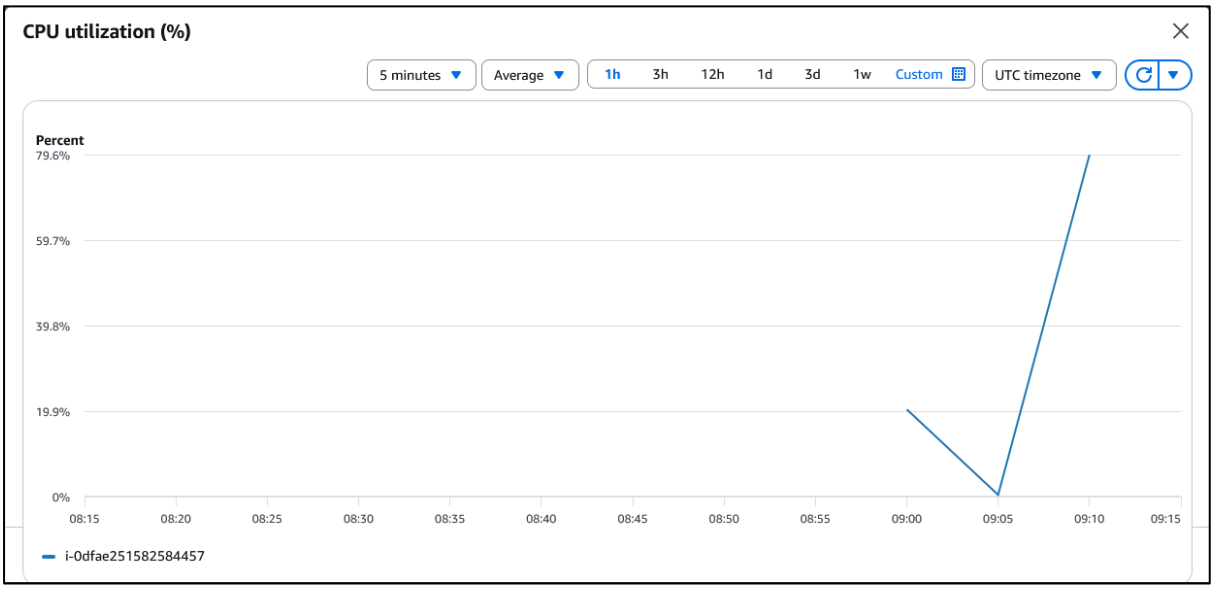
- Run a sustained CPU load test

```
$ stress --cpu 2 --timeout 600
```

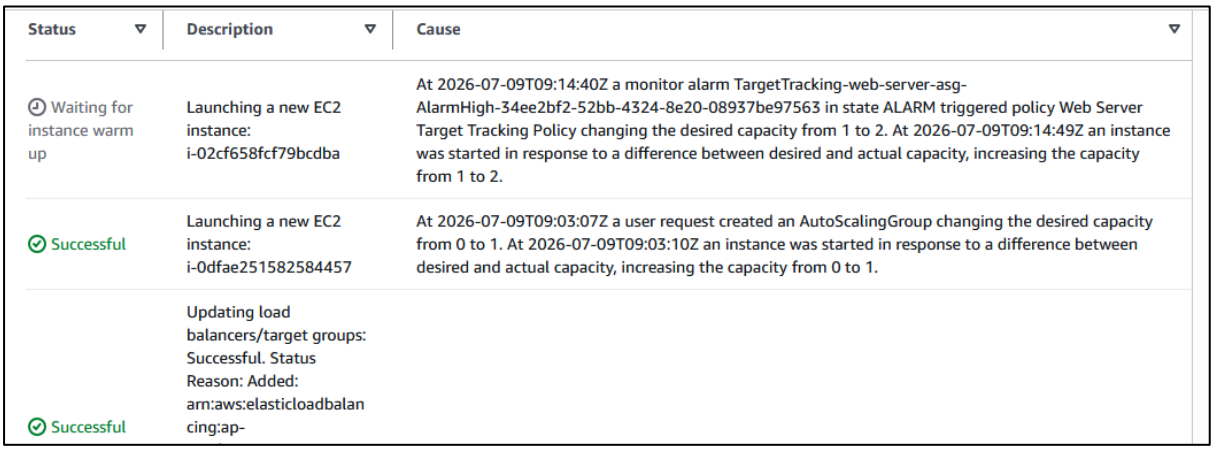
```
[ec2-user@ip-172-31-20-169 ~]$ stress --cpu 2 --timeout 600
stress: info: [26733] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

Step 6: Monitor Scaling

- Open the instance's **Monitoring** tab, watch CPU climb toward and past 50%

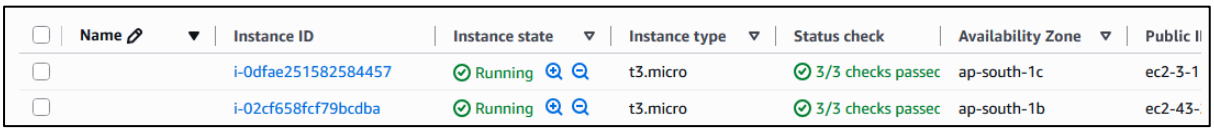


- Open the Auto Scaling Group's **Activity** tab, watch for a scale-out event



Status	Description	Cause
Waiting for instance warm up	Launching a new EC2 instance: i-02cf658fcf79bcdba	At 2026-07-09T09:14:40Z a monitor alarm TargetTracking-web-server-asg-AlarmHigh-34ee2bf2-52bb-4324-8e20-08937be97563 in state ALARM triggered policy Web Server Target Tracking Policy changing the desired capacity from 1 to 2. At 2026-07-09T09:14:49Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 2.
Successful	Launching a new EC2 instance: i-0dfae251582584457	At 2026-07-09T09:03:07Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 1. At 2026-07-09T09:03:10Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 1.
Successful	Updating load balancers/target groups: Successful. Status Reason: Added: arn:aws:elasticloadbalancing:ap-	

- Watch desired capacity increase (1 → 2)

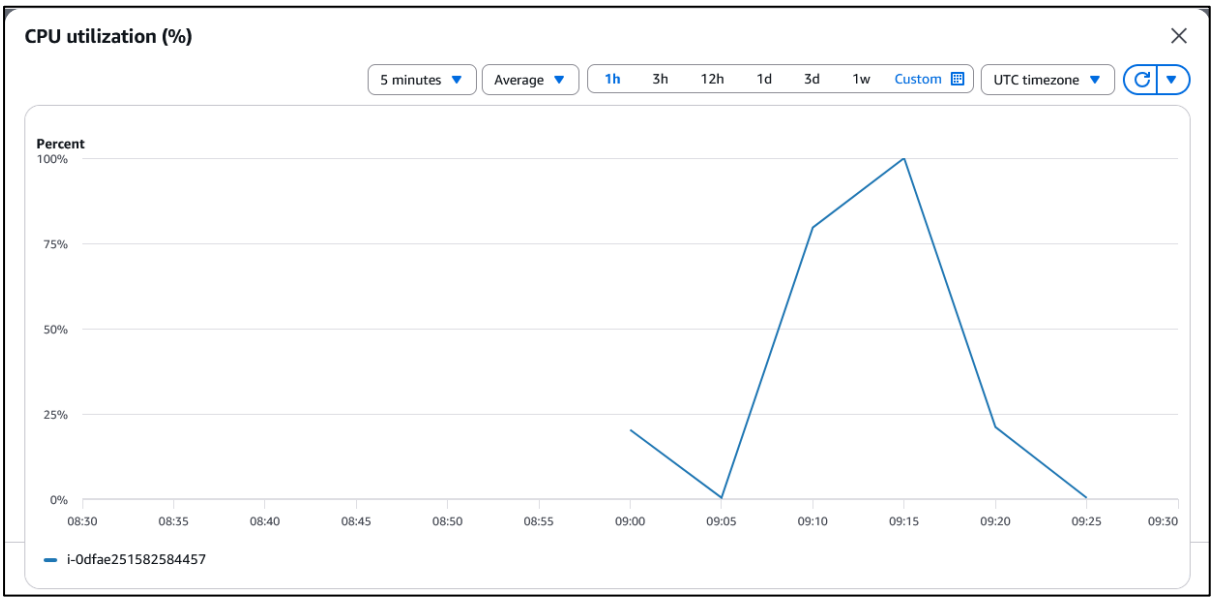


Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public I
	i-0dfae251582584457	Running	t3.micro	3/3 checks passed	ap-south-1c	ec2-3-1
	i-02cf658fcf79bcdba	Running	t3.micro	3/3 checks passed	ap-south-1b	ec2-43-

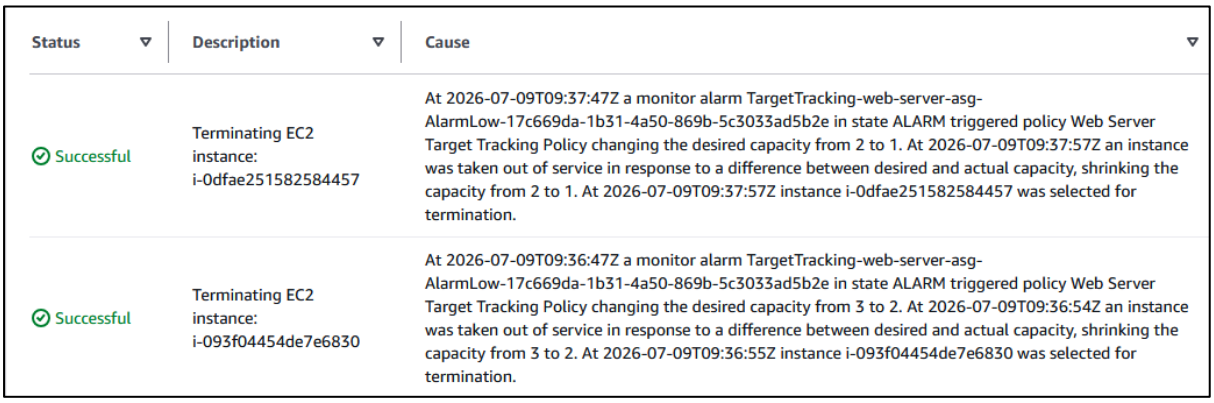
Note: *scaling isn't instant, AWS waits for the metric to stay above target for a sustained period, then applies a cooldown period before evaluating again.*

Step 7: Verify Scale-In

- Stop the stress test (or let the timeout expire)
- Wait for CPU to drop back to baseline on the Monitoring tab

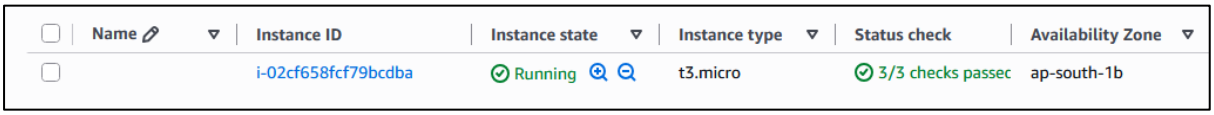


- Open the **Activity** tab, watch for a scale-in event



Status	Description	Cause
Successful	Terminating EC2 instance: i-0dfae251582584457	At 2026-07-09T09:37:47Z a monitor alarm TargetTracking-web-server-asg-AlarmLow-17c669da-1b31-4a50-869b-5c3033ad5b2e in state ALARM triggered policy Web Server Target Tracking Policy changing the desired capacity from 2 to 1. At 2026-07-09T09:37:57Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 2 to 1. At 2026-07-09T09:37:57Z instance i-0dfae251582584457 was selected for termination.
Successful	Terminating EC2 instance: i-093f04454de7e6830	At 2026-07-09T09:36:47Z a monitor alarm TargetTracking-web-server-asg-AlarmLow-17c669da-1b31-4a50-869b-5c3033ad5b2e in state ALARM triggered policy Web Server Target Tracking Policy changing the desired capacity from 3 to 2. At 2026-07-09T09:36:54Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 3 to 2. At 2026-07-09T09:36:55Z instance i-093f04454de7e6830 was selected for termination.

- Watch desired capacity drop back down (2 → 1)



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
	i-02cf658fcf79bcdba	Running	t3.micro	3/3 checks passec	ap-south-1b

The Auto Scaling Group automatically removes the extra instance once CPU stays below target long enough, no manual intervention needed.

Production Considerations

Target Tracking is commonly combined with:

- Minimum and maximum capacity limits, so cost stays within a known ceiling even if the policy overshoots
- Scheduled Scaling, to handle predictable traffic patterns (like business hours) in advance, rather than only reacting after load increases
- CloudWatch alarms, to monitor broader application health beyond CPU, such as memory or error rates
- Load Balancer health checks, so traffic is only routed to instances that are actually ready to handle it

Conclusion

Building on the Auto Scaling implementation, this tutorial introduced Target Tracking Scaling to enable dynamic capacity management based on CPU utilization. By automatically scaling EC2 instances in response to demand, the Auto Scaling Group maintained application availability while reducing the risk of unnecessary resource provisioning. This approach represents an adaptive and production-oriented scaling strategy for applications with variable workloads.